

- **PROCESSI di SVILUPPO**

Approccio disciplinato per la costruzione, rilascio e manutenzione del software, definisce chi, cosa, quando e come per il raggiungimento di un obiettivo.

Agili, progetti a pianificazione incrementale, più malleabili.

Basati sul piano, tutte le attività sono pianificate in anticipo.

Modello a cascata, basato sul piano, fasi separate e distinte.

Analisi dei requisiti, Progettazione del sistema e software, Implementazione e test, Integrazione e test di sistema, Funzionamento e manutenzione. Poco flessibile.

Sviluppo incrementale, piano o agile, specifica, sviluppo e validazione alternati.

Iterativo, ripete il processo più volte, Incrementale, cresce di volta in volta, Evolutivo, il feedback di un iterazione va a raffinare quella successiva.

Integrazione e configurazione, piano o agile, assemblato da componenti esistenti.

Riutilizzo del software, riduce costi, rapidità, sicurezza, compromessi sui requisiti.

Scegliere il modello in base alla complessità, team, budget, tempistiche e rischio.

Processo unificato, sviluppo software orientato agli oggetti, basato su iterazioni sovrapponibili che producono una release ognuna. Ideazione, Elaborazione (nucleo), Costruzione e Transizione (completamento)

Scrum, modello agile di sviluppo che tramite uno sprint (2-4 settimane) e un daily scrum permette di procedere rapidamente nello sviluppo e rilascio di un incremento del prodotto.

- **Cascata**: requisiti, progettazione, codice e test vengono svolti in sequenza.
- È rigida: cambiare requisiti durante il progetto può essere costoso.
- **Iterativo**: una parte del software viene sviluppata e migliorata più volte.
- **Incrementale**: nuove funzionalità vengono aggiunte gradualmente.
- **Evolutivo**: il prodotto cambia in base al feedback e a ciò che si impara.
- **Unified Process**: processo iterativo e incrementale orientato ad architettura e rischi.
- UP usa quattro fasi: **Ideazione, Elaborazione, Costruzione, Transizione**.
- **Agile**: insieme di principi basati su collaborazione, feedback e adattamento.
- **Scrum**: framework Agile che organizza il lavoro in cicli brevi chiamati **Sprint**.
- In sintesi: Agile è la filosofia, Scrum è un modo concreto di applicarla.

- **IDEAZIONE**

Stabilire la visione e la fattibilità del progetto, fare valutazioni e studi. (economici, ..)

Proporre un primo workshop dei requisiti e pianificare la prima iterazione.

Tempo di durata minore a qualche settimana.

- **REQUISITI**

Capacità di cui il sistema deve disporre, complete e coerenti.

Intervista dei clienti, workshop dei requisiti tra sviluppatori e clienti, dimostrazioni..

Funzionali descrivono il comportamento delle funzionalità che il sistema deve disporre.
Non funzionali descrivono le proprietà del sistema nel suo complesso, da cui si derivano requisiti funzionali. (sicurezza, affidabilità, prestazioni, ...)

Modello dei casi d'uso, insieme di scenari tipici dell'utilizzo del sistema.

Specifiche supplementari, ciò che non è un caso d'uso.

Glossario, termini significativi del dominio di sistema.

Visione, riassume ad alto livello i requisiti e riporta lo studio economico.

Regole di business, requisiti e politiche da rispettare.

• ATTORI

Entità dotata di un comportamento.

Primario, raggiunge obiettivi usando il sistema.

Finale, vuole che il sistema sia usato per raggiungere i suoi obiettivi.

Supporto, offre un servizio utilizzato nel sistema.

Fuori scena, ha interesse nel comportamento del sistema ma non lo usa.

• CASI d'USO

Scenario, sequenza specifica di azioni e interazioni tra sistema e attori.

Caso d'uso, collezioni di scenari correlati, descrivono attore e sistema per la raggiunta di un obiettivo specifico. Metodologia semplice per la descrizione dei **requisiti funzionali**.

Modello dei casi d'uso, insieme di tutti i casi d'uso scritti, testi descrittivi ed eventuale UML. Si possono identificare elencando gli obiettivi e gli eventi del sistema. Scritti a vari livelli.

Breve, riepilogo conciso relativo allo scenario principale.

Informale, più paragrafi informali di vari scenari.

Dettagliato, tutti i passi e le variazioni nel dettaglio.

Nome, Portata (sistema), Livello (Utente, sottofunzione), Attore Primario, Parti Interessate, Pre-condizioni, garanzia di successo, estensioni, requisiti speciali, variabili e dati, altro...

Include, include un caso d'uso all'interno di un altro.

Extend, usa opzionalmente un caso d'uso all'interno di un altro.

• ELABORAZIONE

Processo che esegue iterazioni brevi guidate dal rischio, implementa e testa in modo incrementale, adatta in base al feedback.

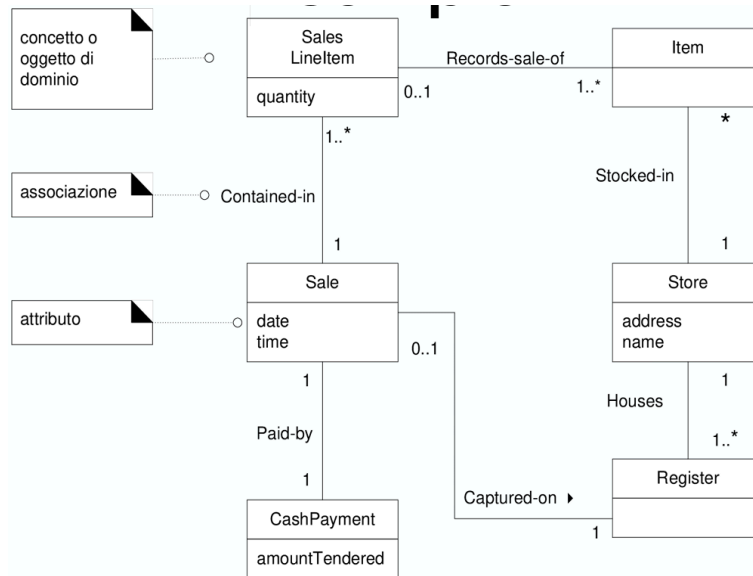
Produce: Modello di Dominio, Modello di Progetto, Documento dell'Architettura Software, Modello dei Dati e la descrizione dell'interfaccia utente.

- **MODELLO di DOMINIO**

Descrive le classi concettuali e le relazioni tra di loro, Ispira le classi di progetto da implementare, Sviluppa iterativamente ed incrementalmente, Limitato dai requisiti dell'iterazione in corso. Descrive i dati persistenti.

Identificabili tramite l'utilizzo di categorie. Elementi, Prodotto o Servizio, Luogo, Ruoli..

Aggregazione, relazioni deboli tra oggetti. **Composizione**, fortemente collegati.

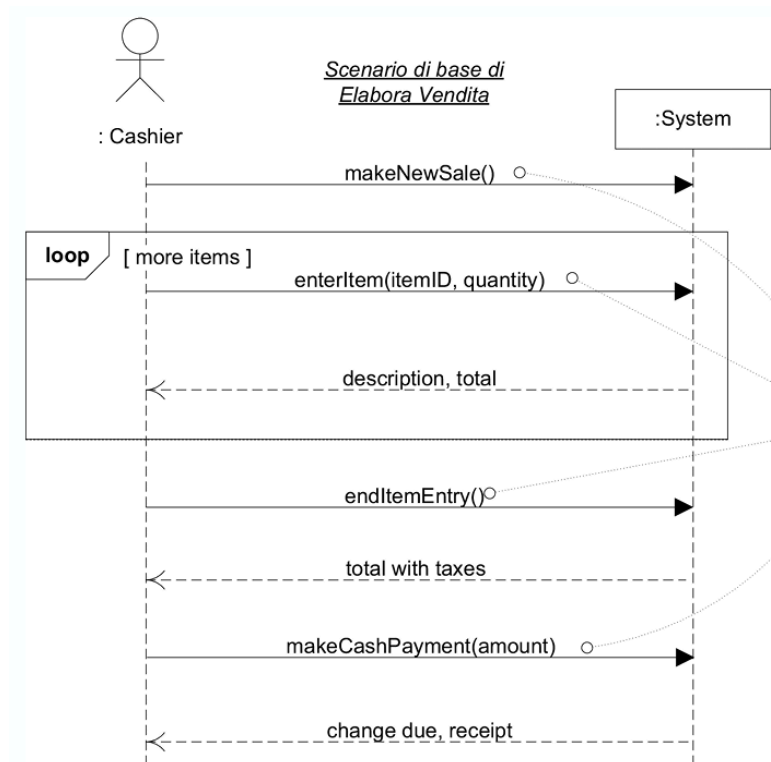


- **DIAGRAMMA di SEQUENZA di SISTEMA - SSD**

Un attore genera **eventi di sistema** per richiedere l'esecuzione di alcune **operazioni di sistema**, operazioni definite dal quest'ultimo per gestire gli eventi, accadimenti degni di nota.

Il diagramma mostra gli eventi generati dagli attori esterni per un particolare scenario e l'ordine degli eventi interni tra i sistemi.

Contratto, descrive il cambiamento dello stato di oggetti del Modello di Dominio e ne indica le Pre-condizioni e le Post-condizioni che devono essere rispettate da un'operazione di sistema.



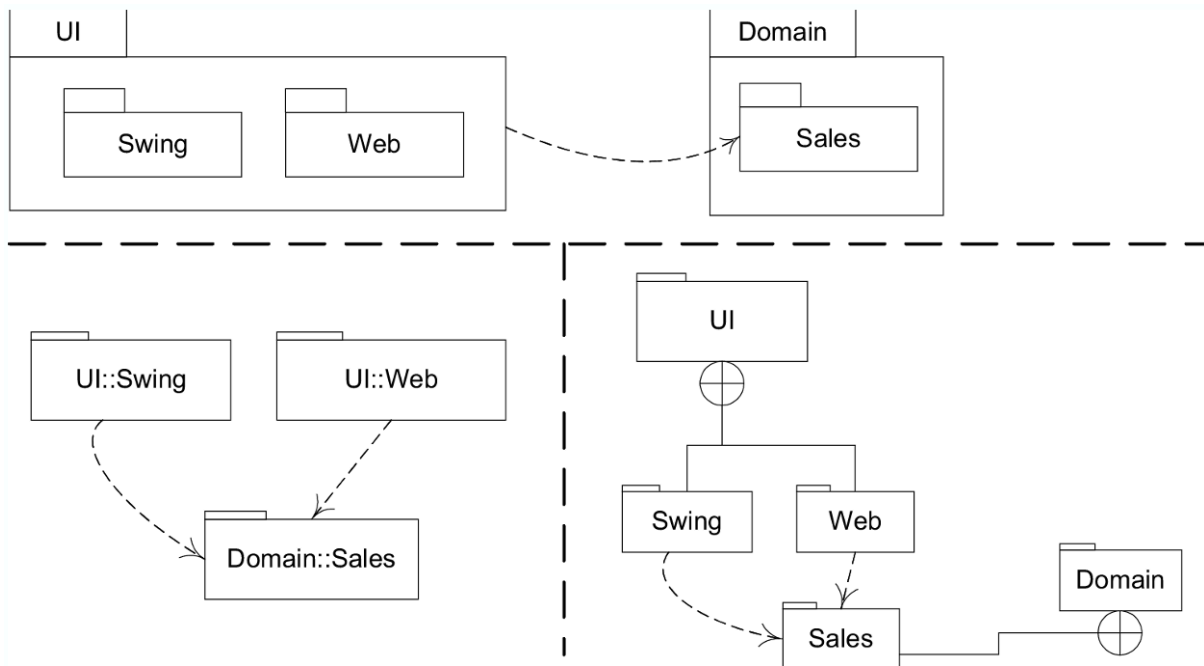
- **ARCHITETTURA LOGICA**

Organizzazione del sistema software su larga scala su classi e package astratti. Generalmente l'SSD lega lo strato di UI a quello di Dominio.

A Strati, gruppo di classi, package o sottoinsieme che ha delle responsabilità in comune con un aspetto rilevante del sistema. Comunemente: **Interfaccia**, applicazione, **dominio**, business, **servizi tecnici**, Foundation.

Strati stretta, uno strato può chiamare solo quello immediatamente sotto.

Strati rilassata, uno strato più alto può chiamare diversi più bassi.



- **DIAGRAMMI di INTERAZIONE**

Illustrano il modo in cui gli oggetti interagiscono fra di loro,

Linee di vita, rappresentano una istanza: **nome [id = x] : tipo**

Messaggi, rappresentano una comunicazione tra due linee di vita.

Di Sequenza, sequenza temporale e ordinata degli scambi di messaggi. (SSD)

Di Comunicazione, mostrano le relazioni strutturali tra gli oggetti.

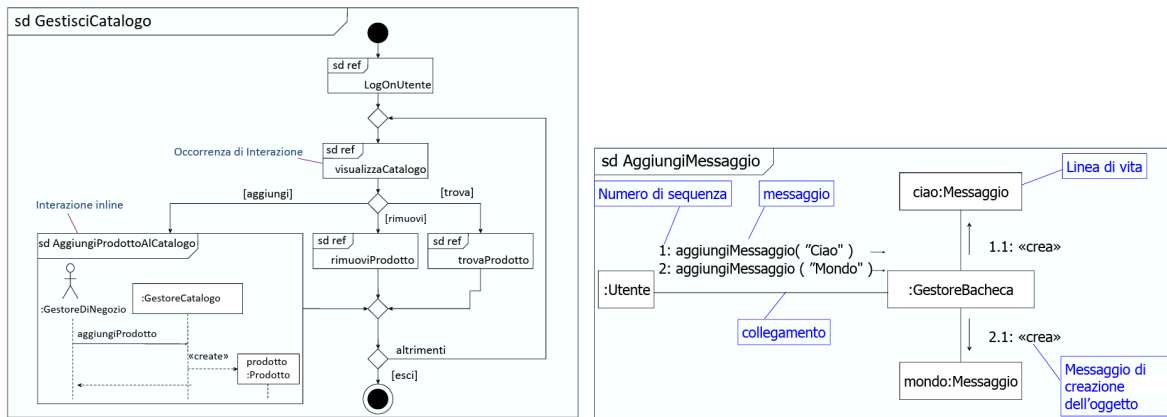
Di Interazione generale, esplodono un comportamento complesso in meno complessi.

Di Temporizzazione, mostrano nel dettaglio aspetti in tempo reale di una interazione.

Possono essere usate condizioni, cicli o iterazioni, numerazione di sequenza, collezioni..

di Interazione generale:

di Comunicazione:



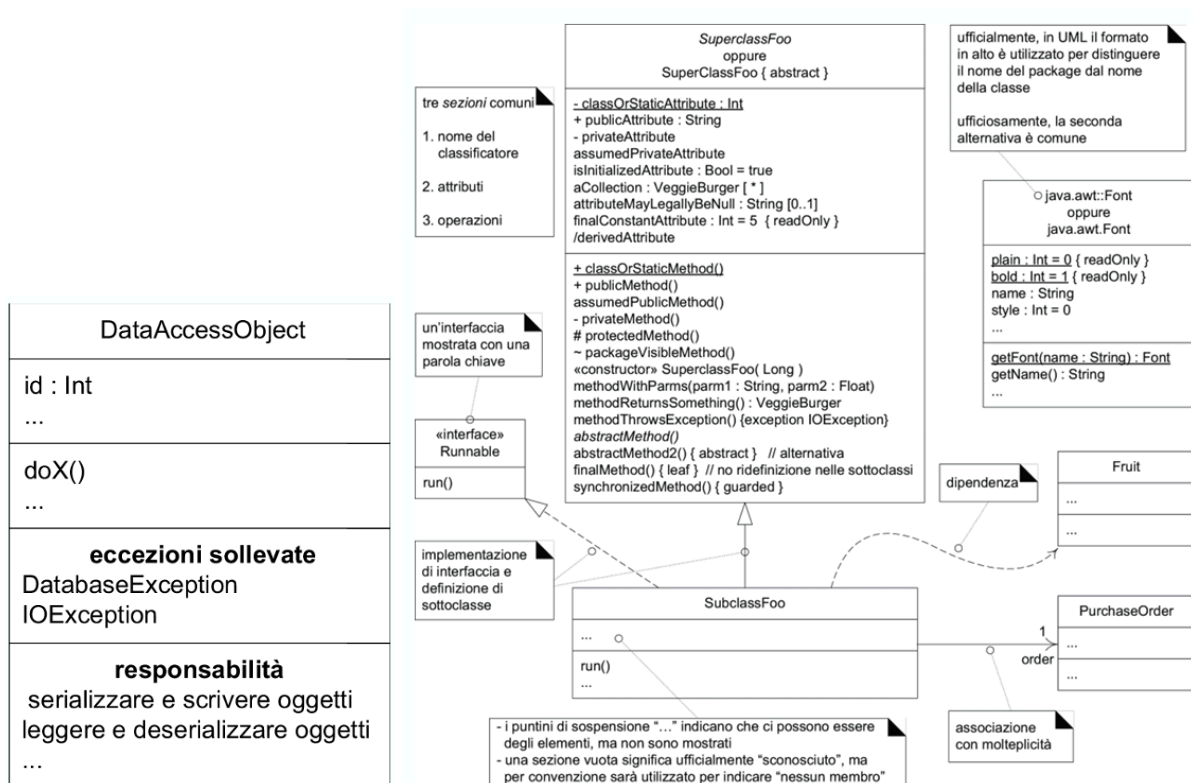
● DIAGRAMMA delle CLASSI

Illustrazione specifica delle classi con relative interfacce, attributi, metodi, associazioni, identificatori e visibilità. (la specifica è detta Classificatore). Valgono le proprietà di java.

Operazioni, definizione di metodi: **visibilità nome(arg, ...) : tipo di ritorno {proprietà}**

Metodo, implementazione di un operazione, aggiunta di tags:

<< create / interface / actor / ... >> / { abstract / ordered / visibility / ... }



● MACCHINE a STATI

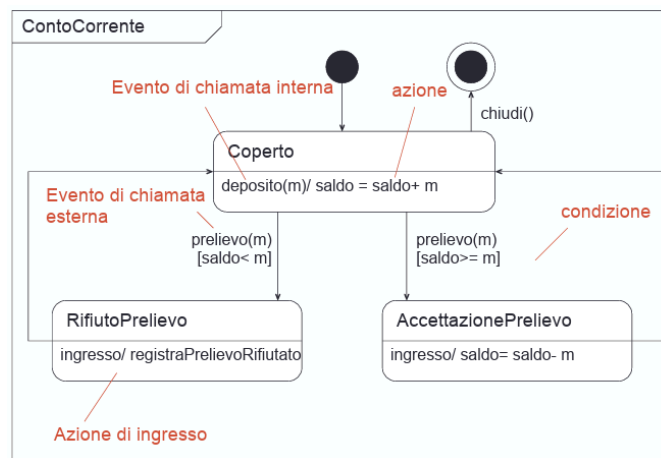
Utilizzati per modellare il comportamento dinamico di classi, casi d'uso e sistemi.
 Rispondono ad eventi, hanno ciclo di vita, cambiano in base agli stati precedenti.
 Possono essere composte da sotto macchine a stati.

Indipendente dallo stato, risponde sempre allo stesso modo ad un evento.

Dipendente dallo stato, risponde a seconda dello stato in cui si trova o alla memoria.

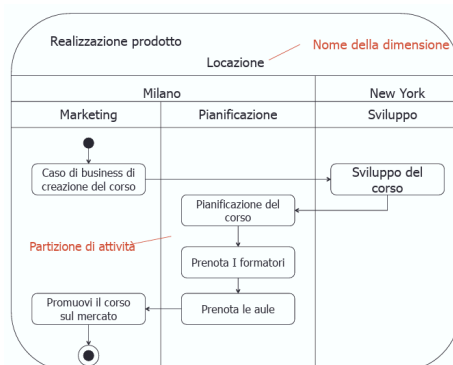
Stato, condizione o situazione di vita di un oggetto che soddisfa una condizione, variano attributi, relazioni e attività di esecuzione.

La memoria può essere semplice se ricorda solo lo stato della macchina oppure multi livello se ricorda anche quella delle sotto macchine o sotto stati.



● DIAGRAMMI dell ATTIVITÀ

Modellano un processo come un'attività costituita da un insieme di nodi connessi da archi
 Attività associate a casi d'uso, classi, interfacce, operazioni, ...



Sintassi	Semantica	Nodi finali
	Nodo iniziale – indica dove inizia il flusso quando invocata un'attività	
	Nodo finale dell'attività – termina un'attività	
	Nodo finale del flusso – termina un flusso specifico all'interno di un'attività. Gli altri flussi non vengono influenzati	
 «inputDecision» Condizione di decisione	Nodo decisione – viene attraverso l'arco in uscita la cui condizione di guardia è soddisfatta	
	Nodo fusione – copia token in ingresso nel suo unico arco in uscita	
	Nodo biforcazione – divide il flusso in più flussi concorrenti	
 {specifica di ricongiunzione}	Nodo ricongiunzione – Sincronizza più flussi concorrenti. Facoltativamente può avere una specifica di ricongiunzione per modificare la sua semantica.	

● PATTERN GRASP

General Responsibility Assignment Software Patterns, danno un nome e principi di base per la progettazione di oggetti assegnando relative responsabilità ad ognuno.

Information Expert, assegna la responsabilità alla classe che possiede le informazioni necessarie a soddisfare quest'ultima.

Creator, assegna la responsabilità di creare una istanza di un'altra classe per contenerla, utilizzarla, registrarla o possederne i dati.

Low Coupling, assegna la responsabilità in modo tale che l'accoppiamento rimanga basso per ridurre l'impatto dei cambiamenti.

Controller, assegna la responsabilità ad un oggetto di rappresentare il sistema complessivo come se fosse una radice, primo oggetto che permette di interagire con il sistema, rappresenta uno scenario di caso d'uso. **Use-Case** se l'interfaccia definisce casi d'uso, **Facade** se espone una facciata del sistema. Progettarlo in modo che esso deleghi a sua volta le operazioni di sistema.

High Cohesion, assegnare le responsabilità in modo che la correlazione fra i vari elementi software rimanga alta, mantenendo il low coupling.

Pure Fabrication, assegnare le responsabilità ad una classe artificiale da cui se ne possono creare altre in modo da mantenere alta coesione, basso accoppiamento e riuso.

Polymorphism, assegnare responsabilità e comportamenti alternativi in base ai tipi di oggetti simili che possono variare.

Indirection, assegna la responsabilità ad un oggetto di fare da intermediario tra altri in modo da impedirne l'accoppiamento diretto.

Protected Variations, si assegnano responsabilità per creare un'interfaccia stabile attorno ai punti in cui sono previste delle variazioni.

- **PATTERN GoF**

I design pattern sono una soluzione progettuale comune ad un problema che ricorre.

Adapter, convertire l'interfaccia originale di un componente in un'altra attraverso un oggetto adattatore che fa da intermediario.

Factory, crea un oggetto Pure Fabrication che gestisce la creazione e inizializzazione degli elementi da esso creati.

Facade, definisce un punto di contatto singolo con il sottosistema, ovvero una facciata di esso. Responsabile della collaborazione con i componenti del sottosistema.

Singleton, definisce un metodo statico della classe che restituisce un oggetto univoco. (sempre quello). Per definire un punto di accesso globale. Spesso utilizzato con i pattern Factory e Facade.

Strategy, definisce una strategia in una classe separata con interfaccia comune.

Composite, definisce le classi per oggetti composti e atomici in modo che implementino la medesima interfaccia, in modo da poter creare un albero.

Observer, definisce un'interfaccia Publish-Subscribe alla quale ci si può iscrivere per ricevere determinati eventi provenienti dai publishers.

- **VISIBILITÀ**

La visibilità tra oggetti può essere ottenuta per **attributo**, per **parametro**, per visibilità **locale** e per visibilità **globale**. Essa può essere trasformata nel passaggio tra i vari oggetti.

- **TESTS, REFACTORING e CODE SMELLS**

Il codice dei test viene scritto prima dell'implementazione, in modo che essa debba soddisfarli. Garantisce maggiore chiarezza, verifica, fiducia e soddisfazione. Necessità di tipologie di input studiate, variabili e sui punti soggetti a condizioni o variazioni.

Si va ad applicare un refactoring qualora si ritenga opportuno effettuare migliorie in evenienza di un aggiunta di funzionalità, correzione di un bug o revisione del codice. Per quest'ultimo si vanno ad applicare alcune metodologie Code Smells.

Long Method, quando un metodo è troppo lungo, lo si estrae usando **Extract Method**, ovvero la creazione di un sotto metodo, andando ad effettuare il passaggio delle variabili come parametri. In evenienza si utilizza **Replace Method with Object** e **Decompose Conditional**, dove si va a sostituire una condizione importante con un metodo richiamabile.

Duplicated Code, si utilizza **Extract Method** per unificare le varie chiamate al codice.

Feature Envy, qualora un oggetto accede più a dati di un'altro rispetto che ai suoi, allora si va ad utilizzare il **Move Method** o l'**Extract Method** per spostare le operazioni all'interno dell'oggetto che gestisce le informazioni.

Large Class, qualora avessimo una classe contenente troppe informazioni o metodi andiamo ad effettuare un **Extract Class** per creare sottoinsiemi di variabili, **Move Method** in altre classi e **Extract Subclass** per creare una sottoclasse che eredita parte del comportamento.

Switch Statement, lo switch case fra tipologie di oggetti può essere sostituito utilizzando **Replace Conditional with Polymorphism**, eventualmente usando **Extract Method** e poi **Move Method**.

Data Class, migrare tutti i metodi di altre classi che nella classe che contiene solo informazioni relative a quest'ultimi con solo setter e getters, si utilizza **Extract Method** oppure **Move Method**.

Long Parameter List, qualora si abbiano troppi parametri in un metodo li si rimpiazzano con un oggetto con **Introduce Parameter Object** oppure passare direttamente oggetti o metodi con **Preserve Whole Object** e **Replace Parameter with Method Call**.

Shotgun Surgery, quando per apportare modifiche è necessario apportare a molte classi diverse si usa **Move Method** e **Move Field** per spostare i comportamenti in una classe che si occupi di quelle determinate responsabilità.

Comment si utilizzano **Extract Variable**, **Extract Method**, **Rename Method**, e **Introduce Assertion** per far sì che il commento spieghi in dettaglio il codice a cui si sta riferendo.

Refused Bequest, quando metodi non necessari rimangono inutilizzati si applica **Replace Inheritance with Delegation** se si tratta di una sottoclasse che eredita metodi senza senso, invece utilizzare **Extract Superclass** se i metodi da ereditare sono opportuni ma non necessari nella sottoclasse.